

FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO

Desenvolvimento Web III

Gabriel Avelar Atanes Batistelo
Gabriel Monteiro Oliveira
Gustavo Dias Bricoli
Pedro Henrique de Souza Jeacomo

**Criação de uma Plataforma Web
Colaborativa de Quebra-Cabeças e Desafios
de Associação Lógica**

Osasco/ São Paulo
2026

Gabriel Avelar Atanes Batistelo
Gabriel Monteiro Oliveira
Gustavo Dias Bricoli
Pedro Henrique de Souza Jeacomio

Criação de uma Plataforma Web Colaborativa de Quebra-Cabeças e Desafios de Associação Lógica

Projeto final, apresentado a Faculdade de Tecnologia do Estado de São Paulo, como parte das exigências para a obtenção de nova da matéria laboratório de desenvolvimento web, ministrado pelo professor Renato de Pierri.

Osasco, 2026.

Código do Projeto: CONECTA-WEB-2026

Versão do Documento: V1.0

Data: 30 de Agosto de 2026

Autor: Equipe de Desenvolvimento de software multiplataforma / TI

1. Resumo gerencial

Este relatório documenta as especificações técnicas, escopo e planejamento para a implementação do Conecta. Uma plataforma web interativa inspirada em jogos de quebra-cabeça e associação de palavras (Como o jogo web Conexo), expandida para uma dinâmica colaborativa de “rede social”. O ambiente foi projetado para transformar uma experiência que tradicionalmente é estática e individual em um ecossistema comunitário, permitindo que os próprios usuários criem, publiquem, resolvam e avaliem desafios lógicos.

A arquitetura do projeto foi estruturada de forma modular, moderna e escalável utilizando Next.js, TypeScript, PostgreSQL e Prisma ORM, viabilizando a entrega incremental de valor por meio de um Produto Mínimo Viável (MVP) sólido e sustentável.

2. Contextualização do problema

Jogos clássicos de associação e raciocínio lógico frequentemente operam sob um modelo centralizado e limitada: os desafios são predefinidos pela equipe desenvolvedora com atualizações periódicas e espaçadas, restringindo o jogador a uma postura passiva de resolução.

Essa limitação restringe a vida útil do engajamento do usuário e impede a personalização de conteúdos temáticos, a colaboração comunitária e a criatividade coletiva. Além disso, a falta de uma infraestrutura com moderação descentralizada e mecanismos de curadoria orientados pela comunidade dificulta a identificação de desafios de alta qualidade e o combate a conteúdos de baixo nível ou repetitivos. O projeto Conecta surge para preencher essa lacuna, integrando mecânicas de lógica a um ambiente dinâmico de “rede social”.

3. Objetivos do projeto

O objetivo principal é desenvolver e implementar uma solução web funcional (MVP) voltada ao entretenimento educativo e comunitário, incentivando o raciocínio lógico e o compartilhamento de conteúdo. Os objetivos específicos subdividem-se em:

- **Automação e Criação de Conteúdo:** Disponibilizar interfaces reativas e intuitivas para a criação, validação estrutural e publicação de quebra-cabeças de associação por grupos de palavras;
- **Interação e Curadoria Comunitária:** Estabelecer módulos de ativação, classificação, favoritos e denúncias para assegurar a relevância e qualidade contínua do catálogo de jogos;
- **Segurança e Persistência Integrada:** Implementar autenticação segura, controle de perfis e integridade relacional de dados com banco de dados relacional e ORM robusto;
- **Visibilidade e Engajamento:** Estruturar rankings, destaques de criadores/puzzles populares e filtros de busca para fácil navegação e fidelização do público.

4. Engenharia de requisitos: Atores e casos de uso

O sistema foi estruturado com base nos seguintes perfis de atores:

- **Ator primário – jogador/ criador:** Usuário cadastrado e autenticado com permissões para resolver desafios, criar e publicar novos puzzles, avaliar criações de comunidade, favoritar jogos e gerenciar seu perfil;
- **Ator secundário – visitante (Não autenticado):** Usuário com permissão de leitura restrita para navegação básica no catálogo público e resolução de quebra-cabeças em modos demonstração;
- **Ator secundário – administrador/moderação:** Responsável pela moderação da plataforma, análise de denúncias de conteúdo inadequado e manutenção da integridade geral do sistema.

4.1 Mapeamento das unidades de caso de Uso

4.1.1 Módulo de Autenticação e segurança

- Cadastrar usuário;
- Autenticar usuário (login/ sessão via Auth.js);
- Gerenciar perfil de usuário.

4.1.2 Módulo de criação e gestão de puzzles

- Criar e publicar puzzle (com validação de grupos e palavras);
- Editar/ excluir puzzle próprio;
- Denunciar conteúdo inadequado ou repetitivo.

4.1.3 Módulo de gameplay e interação social

- Jogar/ resolver quebra-cabeça da comunidade;
- Avaliar e classificar puzzle concluído;
- Adicionar/ remover puzzle dos favoritos;

4.1.4 Módulo de descoberta e rankings

- Buscar e filtrar puzzles (por tema, popularidade ou autor);
- Consultar rankings e puzzles em destaque.

5. Narrativa técnica de caso de uso

Para melhor entendimento das regras de funcionamento, abaixo é detalhado o caso de uso mais crítico do núcleo do sistema:

Caso de uso: Criar e editar puzzle

Ator principal: Jogador/ Criador (autenticado)

Pré-condições: O usuário deve estar previamente autenticado (registrado) na plataforma.

5.1 Fluxo principal:

- O usuário acessa a Plataforma e clica na opção “Criar novo puzzle”;
- A interface renderiza a grade de configuração com campos para título, categorias e palavras associadas;

- criador define os 4 grupos conceituais e insere as 4 palavras correspondentes para cada grupo (totalizando 16 palavras)
- Sistema valida em tempo real se não há termos duplicados, campos em branco ou incoerências estruturais;
- criador clica no comando de visualização previa (preview) e em seguida em publicar desafio;
- A Plataforma persiste as entidades no banco de dados PostgreSQL via Prisma ORM e indexa o novo puzzle no catálogo global;
- Sistema exibe mensagem de sucesso e redireciona o usuário para a página de compartilhamento do puzzle recém-criado;

5.2 Fluxo de Exceção (Dados inválidos/incompletos):

- O usuário tenta publicar o puzzle com palavras duplicadase, menos de 4 grupos ou caracteres não permitidos;
- sistema interrompe a requisição de persistência;
- Uma notificação visual de erro é renderizada na tela com indicação dos campos que necessitam de correção;
- O botão de publicação permanece bloqueado até a total conformidade das regras de valiação.

6. Arquitetura e Stack tecnológica

A aplicação adota uma arquitetura desacoplada e moderna no ecossistema JavaScript/TypeScript

- Front-end/ Framework: Next.js (React + TypeScript) para renderização eficiente e reatividade;
- Back-end & APIs: Serve Actions e Route Handlers integrados no Next.js/ Node.js;

- Persistência de dados: PostgreSQL com Prisma ORM para garantia de tipagem estática e integridade relacional;
- Autenticação: Auth.js (NextAuth) com sessões criptografadas e suporte a múltiplos provedores;
- Infraestrutura e DevOps: Docker para padronização do ambiente de banco de dados e Git/GitHub para versionamento de código

7. Requisitos funcionais (RF)

Os requisitos funcionais descrevem as funcionalidades, comportamentos e serviços que o sistema Conecta deve disponibilizar aos seus usuários, podendo ser verificado através do quadro 1 informando o nome do requisitos funcionais desejáveis, descrição da sua funcionalidade e a prioridade dentro do seu desenvolvimento.

Quadro 1. Requisitos funcionais estimáveis para a plataforma Conecta

ID	Nome	Descrição	Prioridade
RF01	Cadastro e Login de Usuários	O sistema deve permitir que os usuários criem contas, façam login e encerrem sessões com segurança via autenticação por credenciais ou provedores OAuth.	Prioridade
RF02	Gestão de Perfil	O sistema deve permitir ao usuário autenticado visualizar e editar informações básicas de seu perfil (nome de exibição, foto/avatar e bio).	Obrigatório
RF03	Criação de Puzzles	O sistema deve permitir a usuários autenticados criar quebra-cabeças definindo 4 categorias conceituais e 4 palavras/termos por categoria (totalizando 16 itens).	Importante
RF04	Validação Estrutural de Puzzles	O sistema deve validar no momento da criação se há palavras repetidas, campos nulos ou categorias incompletas antes de permitir a persistência.	Obrigatório

RF05	Edição e Exclusão de Puzzles	O criador deve ter permissão para editar ou excluir apenas os puzzles de sua própria autoria.	Obrigatório
RF06	Mecânica de Jogo (Resolução)	O sistema deve renderizar o tabuleiro embaralhado (16 palavras), validar as seleções de 4 itens feitas pelo jogador, computar erros e exibir a vitória ao completar os 4 grupos.	Obrigatório
RF07	Avaliação e Classificação	O sistema deve permitir que jogadores avaliem um puzzle com nota/estrelas (ex: de 1 a 5) e feedback rápido após a conclusão da partida.	Obrigatório
RF08	Sistema de Favoritos	O usuário autenticado deve conseguir salvar puzzles em uma lista pessoal de "Favoritos" para jogar ou acessar posteriormente.	Importante
RF09	Busca e Filtragem de Puzzles	O sistema deve permitir buscar quebra-cabeças por título, categoria temática, nível de dificuldade aparente ou autor.	Desejável
RF10	Rankings e Destaques	O sistema deve gerar listagens automáticas dos puzzles mais jogados, mais bem avaliados e lançamentos recentes na página inicial.	Importante
RF11	Moderação e Denúncias	O sistema deve permitir que os jogadores denunciem puzzles com conteúdo ofensivo, inadequado ou de baixa qualidade para revisão administrativa.	Importante

Quadro 1 - Requisitos funcionais e estimáveis para a plataforma conecta. Acervo pessoal, 2026.

8. Requisitos Não Funcionais (RNF)

Os requisitos funcionais especificam critérios de qualidade, desempenho, restrições, técnicas, segurança e usabilidade da aplicação podendo ser verificado através do quadro 2 informando o nome do requisitos não funcionais desejáveis, descrição da sua funcionalidade e a prioridade dentro do seu desenvolvimento.

Quadro 2. Requisitos Não funcionais estimáveis para a plataforma Conecta

ID	Nome	Descrição	Prioridade
RNF01	Tecnologia & Arquitetura	O sistema deve ser desenvolvido utilizando Next.js (React + TypeScript) no front-end/back-end integrado, com persistência em banco de dados relacional PostgreSQL via Prisma ORM.	Obrigatório
RNF02	Segurança	As senhas dos usuários devem ser armazenadas utilizando algoritmos de hash seguros (como bcrypt/Argon2). A gestão de sessões deve ser controlada via Auth.js com cookies criptografados (HTTPS/SameSite/HttpOnly).	Obrigatório
RNF03	Desempenho & Tempo de Resposta	O tempo de carregamento da interface inicial e a validação de agrupamentos em partida não devem ultrapassar 2 segundos em conexões padrão de banda larga.	Importante
RNF04	Responsividade & Usabilidade	A interface gráfica deve ser totalmente responsiva, adaptando-se a telas desktop, tablets e smartphones, mantendo ergonomia para toques em telas sensíveis.	Obrigatório
RNF05	Integridade de Dados & Concorrência	O banco de dados PostgreSQL deve garantir propriedades ACID para evitar inconsistências em avaliações simultâneas e integridade referencial nas exclusões em cascata.	Obrigatório
RNF06	Ambiente & Containerização	O ambiente de banco de dados e dependências locais deve ser padronizado via contêineres Docker (docker-compose) para garantir paridade entre os ambientes dos desenvolvedores.	Importante
RNF07	Acessibilidade	O sistema deve seguir boas práticas de contraste de cores, suporte a navegação por teclado e semântica HTML para leitores de tela nas interações principais do jogo.	Desejável
RNF08	Manutenibilidade & Versionamento	O código-fonte deve seguir convenções de tipagem estrita com TypeScript, separação de responsabilidades em componentes modulares e versionamento estruturado via Git/GitHub.	Obrigatório

Quadro 2 - Requisitos não funcionais e estimáveis para a plataforma conecta. Acervo pessoal, 2026.